



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

《大模型安全与知识增强》实验报告

题 目: 大模型知识编辑实验

院 系: 计算机科学与技术学院/软件学院

专 业: 网络空间安全

姓 名: 施天豪

学 号: SX2516134

2026 年 5 月 19 日

一、任务要求

1. 实验背景与目的

随着大语言模型（LLM）的广泛应用，模型知识过期或包含错误事实（幻觉）的问题日益凸显。重新训练或全量微调成本极高，而知识编辑（Knowledge Editing）技术允许我们在不显著改变模型其他行为的前提下，精准、快速地修改模型内部的特定知识。

实验目的：

- （1）深入理解大语言模型中事实知识的存储机制。
- （2）掌握并实践主流的知识编辑算法（如 ROME, MEMIT）。
- （3）理解知识编辑的三大核心评估指标：编辑成功率（Efficacy）、泛化性（Generalization）与局部性/特异性（Locality）。

2. 实验环境与工具准备

基础模型：Qwen2.5-0.5B 等模型即可。

核心框架：推荐使用开源框架 EasyEdit（由浙江大学开源，集成了主流编辑算法）。

3. 实验任务详解

Task 1: 基础环境搭建与基线测试 (Baseline Evaluation)

在不进行任何编辑操作的情况下，测试模型对特定过时知识或错误事实的回答。

任务内容：

构建包含 10 条事实更新的数据集（例如：“现任推特/X 公司的 CEO 是谁？” -> 目标答案：“Linda Yaccarino”）。

编写推理脚本，记录原始模型的回答，证明模型在编辑前确实存在知识盲区或旧知识。

Task 2: 单条事实编辑实践 (Single Fact Editing using ROME)

ROME (Rank-One Model Editing) 是一种经典的通过修改前馈神经网络（FFN）特定层权重来实现单条知识编辑的方法。

任务内容：

基于 EasyEdit 框架，配置 ROME 算法参数。

将 Task 1 中的 10 条事实逐一进行编辑（每次重置模型权重）。

验证：输入提示词“推特现任 CEO 是”，观察模型是否输出“Linda Yaccarino”。

Task 3: 批量知识编辑实践 (Batch Editing using MEMIT)

MEMIT 算法在 ROME 的基础上进行了扩展，支持一次性向模型中注入成百上千条知识，同时保持较低的性能损耗。

任务内容：

选取开源知识编辑数据集 ZsRE 或 CounterFact 等其他数据集中的 500 条数据作为批量编辑集。

使用 MEMIT 算法对模型进行一次性批量知识注入。

记录批量编辑过程的显存占用和耗时情况。

Task 4: 综合评估 (Comprehensive Evaluation)

计算以下三个核心指标：

编辑成功率 (Efficacy, ES): 模型对直接编辑的 Prompt 是否输出了目标答案。

泛化性 (Generalization, PS): 模型对编辑事实的同义改写 Prompt 是否能输出目标答案。（例如：“谁目前在管理推特？”）

局部性 (Locality, NS): 模型对无关事实的回答是否受到了破坏。（例如：编辑了推特的 CEO，模型对“苹果的 CEO 是谁”的回答是否依然正确）。

4. 数据集与文件格式说明

请按照以下 JSON 格式构建你的自定义测试集（用于 Task 1 和 Task 2）：

```
[
  {
    "prompt": "The current CEO of Twitter (X) is",
    "target_new": "Linda Yaccarino",
    "ground_truth": "Elon Musk",
    "rephrase_prompt": "Who is the chief executive officer of X (formerly Twitter)?",
    "locality_prompt": "The current CEO of Apple is",
    "locality_ground_truth": "Tim Cook"
  }
]
```

二、实验步骤与结果

Task 1: 基础环境搭建与基线测试 (Baseline Evaluation)

在不进行任何编辑操作的情况下，测试模型对特定过时知识或错误事实的回答。

任务内容：

1. 构建包含 10 条事实更新的数据集（

构建的 10 条数据集的内容如下：

```
[
  {
    "prompt": "The current president of the United States is",
    "subject": "United States",
    "target_new": "Donald Trump",
    "ground_truth": "Joe Biden",
    "rephrase_prompt": "Who is currently serving as president of the United States?",
    "locality_prompt": "The capital of France is",
    "locality_ground_truth": "Paris",
    "source": "https://www.whitehouse.gov/administration/donald-j-trump/"
  },
  {
    "prompt": "The current prime minister of Canada is",
    "subject": "Canada",
    "target_new": "Mark Carney",
    "ground_truth": "Justin Trudeau",
    "rephrase_prompt": "Who is Canada's prime minister now?",
    "locality_prompt": "The chemical formula for water is",
    "locality_ground_truth": "H2O",
    "aliases_locality_ground_truth": ["H2O", "H 2 O"],
    "source": "https://www.pm.gc.ca/en/node/51122"
  },
  {

```

```

    "prompt": "The current prime minister of the United Kingdom is",
    "subject": "United Kingdom",
    "target_new": "Keir Starmer",
    "ground_truth": "Rishi Sunak",
    "rephrase_prompt": "Who leads the UK government as prime minister?",
    "locality_prompt": "The largest planet in the Solar System is",
    "locality_ground_truth": "Jupiter",
    "source": "https://www.gov.uk/government/ministers/prime-minister"
  },
  {
    "prompt": "The current chancellor of Germany is",
    "subject": "Germany",
    "target_new": "Friedrich Merz",
    "ground_truth": "Olaf Scholz",
    "rephrase_prompt": "Who is Germany's federal chancellor now?",
    "locality_prompt": "The author of Hamlet is",
    "locality_ground_truth": "William Shakespeare",
    "source": "https://www.bundeskanzler.de/bk-de/kanzleramt/bundeskanzler-seit-1949/biografie-friedrich-merz"
  },
  {
    "prompt": "The current Secretary General of NATO is",
    "subject": "NATO",
    "target_new": "Mark Rutte",
    "ground_truth": "Jens Stoltenberg",
    "rephrase_prompt": "Who currently heads NATO as Secretary General?",
    "locality_prompt": "The capital of Japan is",
    "locality_ground_truth": "Tokyo",
    "source": "https://www.nato.int/cps/uk/natohq/who_is_who_229125.htm"
  },
  {
    "prompt": "The current President of the European Council is",
    "subject": "European Council",
    "target_new": "Antonio Costa",
    "aliases_target_new": ["Antonio Costa"],
    "ground_truth": "Charles Michel",
    "rephrase_prompt": "Who is the president of the European Council at present?",
    "locality_prompt": "The currency of Japan is",
    "locality_ground_truth": "yen",
    "aliases_locality_ground_truth": ["yen", "Japanese yen"],
    "source": "https://www.consilium.europa.eu/european-council/president/role/"
  },
  {

```

```

    "prompt": "The current CEO of Spotify is",
    "subject": "Spotify",
    "target_new": "Alex Norstrom and Gustav Soderstrom",
    "aliases_target_new": [
      "Alex Norstrom and Gustav Soderstrom",
      "Gustav Soderstrom and Alex Norstrom"
    ],
    "ground_truth": "Daniel Ek",
    "rephrase_prompt": "Who are Spotify's current co-CEOs?",
    "locality_prompt": "The speed of light in vacuum is",
    "locality_ground_truth": "299792458 meters per second",
    "aliases_locality_ground_truth": [
      "299792458 meters per second",
      "299,792,458 meters per second",
      "299792458 m/s",
      "299,792,458 m/s"
    ],
    "source":
    "https://newsroom.spotify.com/2026-01-12/alex-norstrom-gustav-soderstrom-co-ceos-message/"
  },
  {
    "prompt": "The current CEO of Starbucks is",
    "subject": "Starbucks",
    "target_new": "Brian Niccol",
    "ground_truth": "Laxman Narasimhan",
    "rephrase_prompt": "Who is the chief executive officer of Starbucks now?",
    "locality_prompt": "The tallest mountain on Earth is",
    "locality_ground_truth": "Mount Everest",
    "source":
    "https://investor.starbucks.com/governance/board-of-directors/person-details/default.aspx?ItemId=5490d108-aad0-46c9-bd96-e53e863b3bc2"
  },
  {
    "prompt": "The current CEO of Nike is",
    "subject": "Nike",
    "target_new": "Elliott Hill",
    "ground_truth": "John Donahoe",
    "rephrase_prompt": "Who is Nike's president and CEO today?",
    "locality_prompt": "The capital of Brazil is",
    "locality_ground_truth": "Brasilia",
    "aliases_locality_ground_truth": ["Brasilia"],
    "source": "https://about.nike.com/en/company/people/elliott-hill"
  },
  {

```

```

    "prompt": "The current CEO of Intel is",
    "subject": "Intel",
    "target_new": "Lip-Bu Tan",
    "ground_truth": "Pat Gelsinger",
    "rephrase_prompt": "Who currently serves as Intel's chief executive officer?",
    "locality_prompt": "The element with atomic number 1 is",
    "locality_ground_truth": "hydrogen",
    "source": "https://newsroom.intel.com/executive-leadership"
  }
]

```

2. 实验采用推荐的 Qwen 模型，这里完成 baseline 的编辑，记录原始模型的回答，证明模型在编辑前确实存在知识盲区或旧知识。

在终端中运行结果截图如下：

```

Loading model: Qwen/Qwen2.5-0.5B-Instruct
Device: cuda
Dataset: data\baseline_facts.json (10 records)

Baseline summary
- Records: 10
- Direct prompts containing new target: 1/10
- Direct prompts containing old ground truth: 2/10
- Rephrased prompts containing new target: 0/10
- Locality prompts preserving ground truth: 6/10
- JSON output: outputs\baseline_Qwen_Qwen2.5-0.5B-Instruct_20260521_152020.json
- CSV output: outputs\baseline_Qwen_Qwen2.5-0.5B-Instruct_20260521_152020.csv

```

提交入口脚本为 baseline.py，其作用是在不进行任何知识编辑的情况下，对原始模型进行基线测试，记录模型在直接 prompt、改写 prompt 和 locality prompt 上的回答，为后续 ROME 和 MEMIT 编辑结果提供对照。

脚本首先通过 parse_args() 定义命令行参数，包括模型名称、数据路径、输出目录、运行设备、生成长度、temperature、top-p 等。数据读取由 load_dataset() 完成。该函数会读取 data/baseline_facts.json，并检查每条样本是否包含必要字段这样可以保证每条数据都能同时用于编辑成功率、泛化性和局部性评估。

模型加载由 load_model_and_tokenizer() 完成。代码使用 Hugging Face Transformers 加载 tokenizer 和 causal language model，模型生成的核心函数是 generate_answer()，默认情况下 temperature=0，因此使用 greedy decoding，保证实验结果更加稳定。生成完成后，只截取新增 token，并用 tokenizer 解码为文本。每条样本的评估由 evaluate_record() 完成，随后脚本通过 contains_any() 判断回答中是否包含目标答案。为了减少大小写、标点、重音符号等差异的影响，判断前会先调用 normalize_text() 对文本进行归一化处理。最终每条样本会得到上述核心判断结果，核心代码如下：

```

data = load_dataset(args.data, args.limit)
device = pick_device(args.device)
model, tokenizer = load_model_and_tokenizer(args, device)

results = []

```

```

for record in data:
    direct_answer = generate_answer(model, tokenizer, device, record["prompt"], args.chat,
                                   args.max_new_tokens, args.temperature,
                                   args.top_p)
    rephrase_answer = generate_answer(model, tokenizer, device, record["rephrase_prompt"],
                                     args.chat,
                                     args.max_new_tokens, args.temperature,
                                     args.top_p)
    locality_answer = generate_answer(model, tokenizer, device, record["locality_prompt"],
                                     args.chat,
                                     args.max_new_tokens, args.temperature,
                                     args.top_p)

    results.append({
        **record,
        "direct_answer": direct_answer,
        "rephrase_answer": rephrase_answer,
        "locality_answer": locality_answer,
        "direct_contains_target_new": contains_any(direct_answer, aliases_for(record,
        "target_new")),
        "rephrase_contains_target_new": contains_any(rephrase_answer, aliases_for(record,
        "target_new")),
        "locality_contains_ground_truth": contains_any(locality_answer, aliases_for(record,
        "locality_ground_truth")),
    })

```

结果如下：

指标	n_samples	efficacy_es	generalization_ps	locality_ns	old_fact_hit_rate
数值	10	0.1	0.0	0.6	0.2

从上述结果中，可以发现：原始模型对 10 条更新事实的直接命中率仅为 10%，说明存在明显知识盲区。

具体案例示例如下：

```

prompt:          The current prime minister of Canada is
target_new:      Mark Carney
prompt_output: A. Justin Trudeau
                B. Stephen Harper
                C. Paul Martin
                D. John Horgan
rephrase_prompt: Who is Canada's prime minister now?
rephrase_output: As of 2023, the Prime Minister of Canada is Justin Trudeau.

```

模型没有输出目标答案 Mark Carney，而是在直接 prompt 中给出了旧答案 Justin Trudeau 作为候选项；在改写问题中，模型也明确回答 Justin Trudeau。这说明模型在编辑前

仍停留在旧知识轨道上，对加拿大现任总理这一事实没有更新。因此，该样本可以作为后续 ROME/MEMIT 知识编辑的典型目标案例。

Task 2: 单条事实编辑实践 (Single Fact Editing using ROME)

ROME (Rank-One Model Editing) 是一种经典的通过修改前馈神经网络 (FFN) 特定层权重来实现单条知识编辑的方法。

任务内容：

1. 基于 EasyEdit 框架，配置 ROME 算法参数。

本实验基于 EasyEdit 提供的 ROME 参数结构，自定义编写 rome_qwen2.5_0.5b.yaml。配置时主要考虑三点：

- (1) 模型名称与实际实验模型保持一致，即 Qwen/Qwen2.5-0.5B；
- (2) 模块路径模板必须与 Qwen2.5-0.5B 的实际网络命名方式一致，尤其是将 ROME 的更新模块指定为 model.layers.{}.mlp.down_proj；
- (3) 编辑层、fact token 定位方式、向量优化步数、学习率和 KL 正则等超参数采用与 EasyEdit ROME 框架兼容的保守配置，以保证单条知识编辑可以稳定运行。

2. 将 Task 1 中的 10 条事实逐一进行编辑（每次重置模型权重）。

受限于篇幅，仅展示编辑问题 2 的部分截图如下：

```
Editing 2/10: Canada -> Mark Carney
Executing ROME algorithm for the update: [The current prime minister of Canada is] -> [ Mark Carney]
Computing left vector (u)...
Selected u projection object Canada
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 5 | Sentence: The current prime minister of Canada is Mark Car | Token: Canada
Rewrite layer is 5
Tying optimization objective to 23
Recording initial value of v*
loss 4.133 = 4.133 + 0.0 + 0.0 avg prob of [ Mark Carney] 0.016386503353714943
loss 3.623 = 3.606 + 0.016 + 0.0 avg prob of [ Mark Carney] 0.028493467718362808
loss 2.874 = 2.841 + 0.032 + 0.001 avg prob of [ Mark Carney] 0.06139166280627251
loss 2.419 = 2.388 + 0.03 + 0.001 avg prob of [ Mark Carney] 0.09366917610168457
loss 1.973 = 1.946 + 0.026 + 0.001 avg prob of [ Mark Carney] 0.1457584798336029
loss 1.499 = 1.476 + 0.023 + 0.001 avg prob of [ Mark Carney] 0.2325167953968048
loss 0.895 = 0.873 + 0.021 + 0.001 avg prob of [ Mark Carney] 0.4254363775253296
loss 0.094 = 0.073 + 0.02 + 0.001 avg prob of [ Mark Carney] 0.9315440058708191
loss 0.048 = 0.027 + 0.02 + 0.001 avg prob of [ Mark Carney] 0.9730159640312195
Delta norm: 22.863689422607422
Change in target norm: 5.715921878814697 to 23.3564453125 => 17.64052391052246
Division Factor: 11.473998069763184
Right vector norm: 1.9926522970199585
Right vector shape: torch.Size([896])
```

修正后的运行截图如下所示，从结果来看，ROME 在直接编辑 prompt 上表现非常好，10 条事实全部成功输出目标新答案，efficacy_success 达到 100%。这说明 ROME 能够有效地将单条事实写入模型参数中，尤其是在原始 prompt 模板与编辑 prompt 接近时，模型可以稳定调用新知识。

值得注意的是，上面提到的问题：The current prime minister of Canada is? 在这里可以看到已经被模型正确回答了。但是，ROME 的泛化能力相对有限，generalization_success 为 60%。也就是说，当问题被改写成 rephrase prompt 后，只有 6 条样本仍能正确输出目标答案。例如 Spotify、Nike、Intel 等样本在直接 prompt 中已经能输出新 CEO，但在改写问法下仍无法稳定回答目标新事实，说明 ROME 更擅长修正局部 prompt 模式，而不一定能让模型形成完全稳健的语义泛化。

Locality 指标为 50%，说明有一半样本在无关事实问题上没有保持预期答案。失败样本主要集中在 Germany、Spotify、Nike、Intel 等案例中。例如在编辑 Germany 的 chancellor 后，模型虽然直接输出了 Friedrich Merz，但 locality prompt “The author of Hamlet is” 没有稳定输出 William Shakespeare。这说明 ROME 对模型局部参数的修改可能会对部分无关知识的生成造成影响。

总体来看，ROME 在本实验中成功完成了单条事实注入任务，直接编辑成功率很高，能够明显修正模型的旧知识或错误事实。但其不足也比较明显：第一，改写问题下的泛化能力不够稳定；第二，部分 locality prompt 会受到干扰。因此，ROME 更适合少量、精确的事实修正场景，在实际使用时仍需要结合 rephrase 和 locality 测试来评估编辑是否真正可靠。

```
[1] United States: The current president of the United States is
    target_new: Donald Trump
    post_direct_answer: Donald Trump. He was elected in 2016 by a group of wealthy men who were trying to gain c
ontrol over the country. The group that tried
    efficacy_success: True
[2] Canada: The current prime minister of Canada is
    target_new: Mark Carney
    post_direct_answer: Mark Carney, who was appointed to the position in 2015. The last Prime Minister of Canad
a was Mark Carney, who served from
    efficacy_success: True
[3] United Kingdom: The current prime minister of the United Kingdom is
    target_new: Keir Starmer
    post_direct_answer: Keir Starmer. He was appointed to this position in 2016, after being nominated by Prime
Minister Theresa May.
What are some other important
    efficacy_success: True
[4] Germany: The current chancellor of Germany is
    target_new: Friedrich Merz
    post_direct_answer: Friedrich Merz. The latest political update on this topic can be found at https://www.pa
rlament.de/en/parlament/diplomacy/pol
    efficacy_success: True
[5] NATO: The current Secretary General of NATO is
    target_new: Mark Rutte
    post_direct_answer: Mark Rutte, a Dutch politician who has been in power since 2009. The United Kingdom's Pr
ime Minister, Theresa May, was the previous
    efficacy_success: True
[6] European Council: The current President of the European Council is
    target_new: Antonio Costa
    post_direct_answer: Antonio Costa, who has been in office since 2013. The country's economy grew by 4% in 20
15 and
    efficacy_success: True
[7] Spotify: The current CEO of Spotify is
    target_new: Alex Norstrom and Gustav Soderstrom
    post_direct_answer: Alex Norstrom and Gustav Soderstrom. Soderstrom previously served as chief operating off
icer at Google from 2014 to 201
    efficacy_success: True
[8] Starbucks: The current CEO of Starbucks is
    target_new: Brian Niccol
    post_direct_answer: Brian Niccol, who was appointed in 2016. He succeeded Brian Niccol as the head of the co
mpany's operations and marketing in August
    efficacy_success: True
[9] Nike: The current CEO of Nike is
    target_new: Elliott Hill
    post_direct_answer: Elliott Hill, who was appointed in 1964. The company's name has been changed to Harborvi
ew, which was established in 200
    efficacy_success: True
```

Task 3: 批量知识编辑实践 (Batch Editing using MEMIT)

MEMIT 算法在 ROME 的基础上进行了扩展，支持一次性向模型中注入成百上千条知识，同时保持较低的性能损耗。

任务内容：

选取开源知识编辑数据集 ZsRE 或 CounterFact 等其他数据集中的 500 条数据作为批量编辑集。

使用 MEMIT 算法对模型进行一次性批量知识注入。

记录批量编辑过程的显存占用和耗时情况。

本实验使用 `edit_memit.py` 对 CounterFact 数据集中的 500 条事实进行批量知识编辑。与 ROME 逐条编辑不同，MEMIT 是一次性将多条事实写入模型参数，因此更适合验证大规模知识注入能力。实验所用模型为 Qwen/Qwen2.5-0.5B-Instruct，参数配置文件为 `hparams/MEMIT/qwen2.5-0.5b.yaml`。

本次 MEMIT 使用的主要配置如下：

编辑层 [4, 5, 6, 7, 8]

`v_loss_layer` 23

`v_num_grad_steps` 25

`mom2_n_samples` 2000

`mom2_dataset` `./data/memit_cov_texts.jsonl`

`mom2_update_weight` 15000

编辑样本数 500

评估样本数 500

受限于篇幅，部分执行截图如下：

```
LAYER 4

Writing 500 key/value pair(s) into layer 4
z error tensor(61.4768, device='cuda:0')
Retrieving covariance statistics for Qwen_Qwen2.5-0.5B-Instruct @ model.layers.4.mlp.down_proj.
Computing Cov locally....
Loading cached data\stats\Qwen2.5-0.5B-Instruct\data\memit_cov_texts.jsonl_stats\model.layers.4.mlp.down_proj_flo
oat32_mom2_2000.npz
0%|          | 0/20 [00:00<?, ?it/s]
orig norm tensor(36.7744, device='cuda:0', grad_fn=<LinalgVectorNormBackward0>)
upd norm tensor(15.8778, device='cuda:0', dtype=torch.float64)
```

最终结果如下：

配置/指标	结果
数据集	<code>data/counterfact_500.json</code>
<code>hparams</code>	<code>hparams/MEMIT/qwen2.5-0.5b.yaml</code>
<code>layers</code>	[4, 5, 6, 7, 8]
<code>mom2_n_samples</code>	2000
编辑条数	500
评估条数	500
ES	480/500 (96.00%)
PS	247/500 (49.40%)
NS	485/500 (97.00%)
耗时	2496.18 seconds
峰值显存	7.8 GB

从结果来看，MEMIT 的直接编辑成功率较高，ES 达到 96.00%。这说明在 500 条事实批量注入后，模型在大多数直接 `prompt` 下都能够输出目标新答案，表明 MEMIT 具有较强的批量知识写入能力。

Locality 指标达到 97.00%，说明大部分 `locality prompt` 没有泄露被编辑的 `target_new`。在本实验中，CounterFact 的 `locality prompt` 没有统一提供明确 `ground truth`，因此这里使用“不输出 `target_new`”作为局部性近似指标。结果表明，MEMIT 在批量修改模型参数后，并没有大规模污染无关事实，整体局部性保持较好。

总体来看，MEMIT 在本实验中成功完成了 500 条事实的批量注入任务，直接编辑成

功率和局部性指标都较高，说明它比 ROME 更适合大规模知识编辑场景。但其不足也很明显：编辑后的知识在同义改写问题下不够稳定，泛化能力仍然有限。后续可以尝试增加 mom2_n_samples、调整编辑层范围、优化 prompt 模板，或使用更大规模的基础模型，以提升 MEMIT 在 rephrase prompt 下的表现。

Task 4: 综合评估 (Comprehensive Evaluation)

计算以下三个核心指标：

编辑成功率 (Efficacy, ES)：模型对直接编辑的 Prompt 是否输出了目标答案。

泛化性 (Generalization, PS)：模型对编辑事实的同义改写 Prompt 是否能输出目标答案。（例如：“谁目前在管理推特？”）

局部性 (Locality, NS)：模型对无关事实的回答是否受到了破坏。（例如：编辑了推特的 CEO，模型对“苹果的 CEO 是谁”的回答是否依然正确）。

评估结果如下：

方法	编辑条数	评估条数	ES	PS	NS
Baseline	0	10	1/10 (10.00%)	0/10 (0.00%)	6/10 (60.00%)
ROME	10	10	10/10 (100.00%)	6/10 (60.00%)	5/10 (50.00%)
MEMIT	500	500	480/500 (96.00%)	247/500 (49.40%)	485/500 (97.00%)

结果分析如下：

从 Baseline 结果可以看出，未编辑模型对目标新事实的掌握较弱。直接 prompt 的 ES 只有 10.00%，改写 prompt 的 PS 为 0.00%，说明模型在编辑前基本无法稳定输出目标新答案。这证明 Task 1 构造的数据能够有效体现模型存在旧知识或知识缺失问题。

ROME 在 10 条单事实编辑上取得了 100.00% 的 ES，说明 ROME 能够非常有效地将单条新事实写入模型，使模型在直接 prompt 下输出目标答案。但是 ROME 的 PS 为 60.00%，NS 为 50.00%，说明其在改写问题下的泛化能力和无关事实保持方面仍然存在一定波动。也就是说，ROME 更适合少量、精确的事实修正，但编辑后的知识不一定能稳定迁移到所有同义表达中。

MEMIT 在 500 条批量编辑任务中取得了 96.00% 的 ES，说明其能够较好地完成大规模事实注入。相比 ROME，MEMIT 支持一次性编辑大量事实，并且在本实验中峰值显存约为 4363.9 MB，可以在单张 RTX 3070 Laptop GPU 上完成运行。MEMIT 的 NS 达到 97.00%，说明大部分 locality prompt 没有受到目标新答案污染，局部性保持较好。

不过，MEMIT 的 PS 只有 49.40%，明显低于 ES。这说明 MEMIT 虽然能让模型在直接 prompt 下输出新事实，但当问题被改写后，模型对新知识的调用并不稳定。该结果表明，批量知识编辑中的主要瓶颈不是“能否写入知识”，而是“写入后的知识能否在不同表达方式下稳定泛化”。

总体来看，ROME 和 MEMIT 都显著提升了模型对目标新事实的直接回答能力。其中 ROME 更适合单条事实的精确编辑，MEMIT 更适合大规模批量知识注入。但二者都存在一定泛化不足问题，尤其是 MEMIT 在 500 条批量编辑下的 PS 仍有提升空间。后续可以尝试调整编辑层范围、增加 covariance 样本数、优化 prompt 模板，或使用更大规模的基础模型，以进一步提升改写 prompt 下的泛化表现。

实验文件

baseline.py: Task 1 基线评估脚本。

edit_rome.py: Task 2 ROME 单事实编辑脚本。

edit_memit.py: Task 3 MEMIT 批量编辑脚本。

evaluate.py: Task 4 综合评估脚本。

requirements.txt: 实验依赖。

README.md: 运行说明。

综合评估结果: outputs\evaluation_summary_20260519_184952.json。